

DDFacet C++ — Documentação do Algoritmo

Implementação educacional em **C++17** do **Algoritmo 1** do pipeline de imageamento DDFacet (Monnier et al., *SiPS IEEE 2022*), usado em **rádio-interferometria**. O foco do projeto é portar o algoritmo (que existe em Python) para C++ sequencial e depois **paralelizá-lo** (OpenMP agora; MPI multi-nó no futuro).

Índice

1. [Objetivo final](#)
2. [Contexto físico: visibilidades e o plano UV](#)
3. [O que é um Measurement Set \(MS\)](#)
4. [Como o MS é lido](#)
5. [Visão geral do algoritmo \(major/minor cycles\)](#)
6. [Cada passo em detalhe](#)
7. [O desafio da apodização \(correção de grid\)](#)
8. [Como foi implementado em C++](#)
9. [Paralelização \(OpenMP\)](#)
10. [Como rodar e validar](#)
11. [Próximos passos](#)

1. Objetivo final

Um **rádio-interferômetro** (um conjunto de antenas, ex.: VLA, LOFAR, MeerKAT) **não mede a imagem do céu diretamente**. Cada par de antenas mede uma **visibilidade**: um único ponto da **Transformada de Fourier** do brilho do céu.

Objetivo do algoritmo: a partir de um conjunto **incompleto e irregular** de amostras da Transformada de Fourier do céu (as visibilidades medidas), **reconstruir a imagem do céu** — ou seja, descobrir **onde estão as fontes de rádio e qual o brilho de cada uma**.

Formalmente, a relação física (equação de medida) é:

$$V(u,v) = \iint I(l,m) \cdot \exp(-2\pi i (u \cdot l + v \cdot m)) \, dl \, dm$$

- $I(l,m)$ = brilho do céu (a **imagem** que queremos — chamada de *sky model* x).
- $V(u,v)$ = visibilidade medida na posição (u,v) do plano de Fourier.
- (l,m) = cossenos diretores (coordenadas angulares no céu).

Ou seja: **visibilidade = Transformada de Fourier do céu**. Se tivéssemos **todos** os pontos (u,v) , bastaria uma FFT inversa. O problema é que o interferômetro só mede **alguns** pontos → reconstrução incompleta → precisamos de um algoritmo iterativo.

2. Contexto físico: visibilidades e o plano UV

- Cada par de antenas forma uma **baseline** (linha de base). A projeção dessa baseline no plano perpendicular à fonte dá as coordenadas (u, v) (em comprimentos de onda); a componente na direção da fonte é w (causa o "w-term", ignorado nesta fase).
- À medida que a Terra gira, cada baseline traça uma elipse no **plano UV** → a cobertura cresce, mas **nunca preenche todo o plano**.
- A cobertura UV incompleta é a **função de amostragem** $S(u,v)$. Sua Transformada de Fourier inversa é o **dirty beam** (a PSF — *Point Spread Function*).

Consequência central: se fizermos a FFT inversa direta das visibilidades, obtemos a **imagem suja** (*dirty image*):

```
imagem_suja = céu_verdadeiro ⊗ dirty_beam      (⊗ = convolução)
```

ou seja, cada fonte pontual aparece "borrada" pela PSF, com lóbulos laterais (*sidelobes*). O trabalho de **deconvolução (CLEAN)** é remover essa PSF e recuperar as fontes pontuais.

3. O que é um Measurement Set (MS)

Um **Measurement Set (MS)** é o **formato-padrão de dados** em rádio-astronomia (formato CASA). Na prática é um **diretório com várias tabelas** (um mini banco de dados) que guarda as visibilidades de uma observação.

A tabela principal (`MAIN`) tem **uma linha por medida**; as colunas relevantes para o imageamento são:

Coluna	Significado
<code>UVW</code>	vetor da baseline (u, v, w) em metros (convertido p/ comprimentos de onda)
<code>DATA</code>	a visibilidade complexa medida $V(u,v)$
<code>FLAG</code>	marca dados inválidos (RFI, antena com problema) — <code>true</code> = descartar
<code>WEIGHT</code>	peso/qualidade da medida
<code>ANTENNA1/2</code> , <code>TIME</code>	par de antenas e instante da medida

Subtabelas associadas: `ANTENNA` (posições), `SPECTRAL_WINDOW` (frequências/canais), `FIELD` (apontamento), `POLARIZATION` , etc.

No algoritmo, o MS corresponde ao índice `j` (`for j in J_MS`). Múltiplos MS aparecem quando há **várias épocas de observação, bandas de frequência ou configurações** do array.

Como o MS é representado no código

A estrutura `MeasurementSetInfo` espelha o MS real:

```
struct MeasurementSetInfo {
    int id; // índice j (0 .. J-1)
    std::string path; // caminho do arquivo MS
    VisibilitySet visibilities; // v_MSj - visibilidades MEDIDAS
    VisibilitySet predicted; // v_MSj - visibilidades PREDITAS pelo modelo
    VisibilitySet residual; // δv_MSj - residual = medido - predito
    double freq_min, freq_max, time_start, time_end;
};
```

E `VisibilitySet` é a tradução direta das colunas do MS:

```
struct VisibilitySet {
    std::vector<double> u, v, w; // coordenadas UVW
    std::vector<std::complex<float>> data; // coluna DATA
    std::vector<float> weight; // coluna WEIGHT
    std::vector<bool> flag; // coluna FLAG
    size_t nvis; // número de visibilidades
    double freq_ref, wavelength;
};
```

4. Como o MS é lido

No DDFacet real (Python): o MS é aberto com a biblioteca `python-casacore` (*casacore tables*), que lê as colunas `UVW`, `DATA`, `FLAG`, `WEIGHT` da tabela `MAIN` e os metadados das subtabelas. Os (u,v,w) em metros são convertidos para comprimentos de onda dividindo pela `wavelength` de cada canal.

Nesta implementação (C++): usamos um **MS real** (formato CASA), com a mesma biblioteca `casacore` do DDFacet de produção — dividido em duas etapas:

(a) Geração — `tools/make_ms.py` (python-casacore):

1. Sorteia a cobertura UV — `n_vis` pontos (u,v) num disco de raio `u_max`, em **pares hermitianos** (u,v) e $(-u,-v)$ (a visibilidade satisfaz $V(-u,-v)=\text{conj}(V(u,v))$ para um céu real $\rightarrow$ imagem reconstruída real).
2. Calcula as visibilidades verdadeiras por **DFT direta** das fontes (equação de medida): ``  $V(u,v) = \sum_{\text{fontes}} A \cdot \exp(-2\pi i (u \cdot l + v \cdot m))$  ``
3. Cria um **Measurement Set de verdade** (`default_ms`), preenche `UVW` (metros), `DATA`, `FLAG`, `WEIGHT` e `SPECTRAL_WINDOW.CHAN_FREQ`, e grava um **sidecar de fontes** (`*.sources.txt`) para a validação no C++.

(b) Leitura — `read_ms` (casacore C++): abre o `.ms`, lê as colunas `UVW` / `DATA` / `FLAG` / `WEIGHT` e a frequência (`CHAN_FREQ[0]`), converte `UVW` de metros para comprimentos de onda ($\div$ `wavelength`) e preenche o `VisibilitySet`. O `main` **gera o MS automaticamente** (chama o python) se ele não existir.

O leitor casacore fica isolado em `src/ms_io.cpp` — só esse arquivo depende do casacore. Trocar por um MS observado de verdade não muda mais nada no pipeline.

5. Visão geral do algoritmo

O Algoritmo 1 é um esquema iterativo de **major cycles** (ciclos maiores) e, dentro da deconvolução, **minor cycles** (ciclos menores do CLEAN).

```

Initialization(v);

for k in K MajorCycles:           # refina o modelo progressivamente
    for j in J MeasurementSets:
        for i in I Facets:        # --- PREDICT ---
             $\hat{g}_{\phi i, MSj}$  = FFT( $\hat{x}$ , i, j);    # modelo → domínio UV
             $\hat{v}_{\phi i, MSj}$  = Degrid( $\hat{g}$ , i, j);    # amostra UV nas posições (u,v)
             $\delta v_{MSj}$  =  $v_{MSj}$  -  $\hat{v}_{MSj}$ ;        # RESIDUAL (medido - predito)
            for i in I Facets:    # --- GRID ---
                 $g_{\phi i, MSj}$  = Grid( $\delta v_{MSj}$ , i, j); # residual → grade UV
            for i in I Facets:    # --- IMAGE ---
                 $\delta y$  += FFT_inv( $g_{\phi i}$ );        # grade UV → imagem suja residual
             $\hat{x}$  = Deconvolution( $\delta y$ , PSF);        # CLEAN: atualiza o modelo do céu

```

A **ideia central** é um laço de **predição e correção**:

1. **Predict**: pegue o modelo atual do céu e calcule *que visibilidades ele produziria*.
2. **Residual**: subtraia das visibilidades medidas → o que o modelo **ainda não explica**.
3. **Image + Deconvolve**: transforme o residual em imagem e use o CLEAN para **adicionar novas fontes ao modelo**.
4. Repita. A cada *major cycle* o modelo melhora e o residual encolhe.

Por que "facetas" (o índice I)?

A imagem é dividida em I **facetas** (regiões). Cada faceta:

- é pequena o suficiente para que aproximações de campo plano valham (corrige o w-term e efeitos direcionais — *Direction-Dependent Effects*);
- tem sua **própria grade UV** e seu **centro de fase** (l_o, m_o);
- é **independente das outras** → unidade natural de **paralelismo**.

O faceamento $I > 1$ funciona **ponta a ponta**: a fase de faceta $\exp(\mp 2\pi i(u \cdot l_o + v \cdot m_o))$ é aplicada no degrid / grid (em função do (u, v) real), o dirty beam e a taper de apodização são computados **por faceta**, e o CLEAN deconvolui cada faceta. Validado: $I=2 \times 2$ recupera 98–102%, $I=4 \times 4$ recupera ~100%. O nº de facetas por eixo é configurável via `DDF_FACETS` (padrão 1).

6. Cada passo em detalhe

6.0 Initialization — initialization

Prepara o estado: aloca o modelo $\hat{x}$ (zerado), a imagem residual δy , a **PSF**, e cria as facetas (`create_facets`: calcula `(offset_x, offset_y)` e o centro (l_o, m_o) de cada faceta). Roda **uma vez**.

6.1 FFT (Predict) — imaging_fft

O que faz: transforma a sub-imagem da faceta (do modelo $\hat{x}$) para o **domínio UV**, produzindo a grade $\hat{g}$ que o Degrid vai amostrar.

Passos:

1. Copia a sub-imagem da faceta i de `state.x` (região `[offset_x..] × [offset_y..]`).

2. Aplica a **rotação de fase** do centro da faceta: $\text{pixel} \cdot= \exp(-2\pi i (l_0 \cdot u_x + m_0 \cdot v_y))$, com $u_x = (ix-cx)/nx$. Isso reposiciona a faceta.
3. `ifftshift` (centra a fase no centro da imagem — necessário p/ a correção de grid).
4. **FFT 2D forward** (via FFTW3).
5. `fftshift` (coloca o DC/frequência-zero no centro — convenção DDFacet).

Resultado em `facet.uv_grid`.

6.2 Degrind — `degrid`

O que faz: a grade UV é **uniforme**, mas as visibilidades estão em posições (u,v) **irregulares**. O Degrind **interpola** (amostra) a grade nessas posições → visibilidades **preditas** $\hat{v}$.

Para cada visibilidade k :

1. Converte (u_k, v_k) para posição em pixels: $ix_c = u_k \cdot nx \cdot cell + nx/2$.
2. Convolui com um **kernel Gaussiano** de suporte $W=5$, $\sigma=1$ pixel: ``  $\hat{v}_k = \sum uv\_grid(ix, iy) \cdot C(du, dv) / \sum C(du, dv)$  ,  $C(du, dv) = \exp(-(du^2 + dv^2)/2\sigma^2)$  ``
3. Acumula em `predicted.data[k]` (soma sobre facetas).

É o operador **forward** (imagem → visibilidades).

6.3 `compute_residual` — `compute_residual`

$$\delta v = v \text{ (medido)} - \hat{v} \text{ (predito)}$$

É o que o modelo atual **ainda não explica**. (Usamos a convenção do Algoritmo 1, $\delta v = v - \hat{v}$, que faz o laço **convergir**: quando o modelo cresce em direção à fonte verdadeira, $\hat{v} \rightarrow v$ e o residual → 0.) A energia $\|\delta v\|^2$ é a métrica de convergência.

6.4 Grid — `grid`

O que faz: operação **adjunta** do Degrind. Distribui cada visibilidade **residual** δv_k de volta na grade UV da faceta, ponderada pelo kernel.

Para cada visibilidade k : `uv_grid(ix, iy) +=  $\delta v_k \cdot \text{conj}(C(du, dv))$` em torno de (ix_c, iy_c) . **Acumula** (vários MS se somam). É o operador **backward** (visibilidades → grade UV).

Detalhe de paralelização: visibilidades vizinhas escrevem nas **mesmas células** → há condição de corrida.
Resolvida com **grade local por thread + redução** (Seção 9).

6.5 `FFT_inv (Image)` — `imaging_fft_inv`

O que faz: converte a grade UV (residual) de volta para o **domínio de imagem** e **acumula** na imagem cuja residual δy .

Passos: `ifftshift` → **IFFT 2D** (normalizada por $1/(nx \cdot ny)$) → `fftshift` → **derotação de fase** (inverso do passo 6.1) → soma a parte **real** em `state.delta_y`.

Ao final do laço de facetas, δy é a **imagem suja do residual** = `residual * dirty_beam`.

6.6 Deconvolution (CLEAN) — deconvolution

O que faz: remove a PSF da imagem suja e **adiciona fontes ao modelo** $\hat{x}$. Usa o algoritmo **Högbom CLEAN** (minor cycles):

```
repete (até n_minor_iterations ou pico < threshold):
  1. acha o pico de  $|\delta y| \rightarrow (\text{peak}_x, \text{peak}_y)$ 
  2.  $\hat{x}(\text{peak}) += \gamma \cdot \delta y(\text{peak})$  # adiciona componente ( $\gamma = \text{clean\_gain}$ )
  3.  $\delta y -= \gamma \cdot \delta y(\text{peak}) \cdot \text{PSF}(\text{centrada no pico})$  # subtrai a PSF escalonada
```

Cada componente CLEAN é uma "fonte pontual candidata". A subtração da PSF remove os *sidelobes* daquele pico, revelando fontes mais fracas. É **inerentemente sequencial** (cada iteração depende da subtração anterior).

Como tudo se encaixa para atingir o objetivo

```
 $\hat{x}$  (modelo) --FFT-->  $\hat{g}$  --Degrid-->  $\hat{v}$  (predito)
                                   |
                                    $\delta v = v - \hat{v}$  |  $v$  (medido, do MS)
                                   v
 $\delta y$  (imagem suja) <--FFT_inv--  $g$  <--Grid--  $\delta v$ 
                                   |
                                   +---CLEAN--->  $\hat{x}$  atualizado → repete o major cycle
```

A cada volta, $\hat{x}$ ganha as fontes que faltavam e $\|\delta v\|^2$ cai. Quando o residual fica no nível do ruído, o **modelo do céu** $\hat{x}$ é a **imagem reconstruída** — o objetivo final.

7. O desafio da apodização (correção de grid)

Este foi o ponto técnico mais delicado da implementação (e é um conceito real de rádio-imageamento).

O kernel Gaussiano de gridding, **no domínio da imagem**, equivale a multiplicar por uma *taper* $T(1, m)$ (a Transformada de Fourier do kernel). Isso **atenua fontes longe do centro de fase** — cada faceta tem, portanto, um **campo de visão útil limitado**. (Esta é justamente uma das motivações do **faceamento**: cada faceta imageia um campo pequeno onde a taper é quase plana.)

Soluções adotadas:

- **FFT centrada** (`ifftshift` / `fftshift` no domínio-imagem) para pôr o centro de fase no centro da imagem, tornando a taper simétrica.
- A imagem suja é escalada por $1/S$ (S = pico do dirty beam); o modelo converge em unidades apodizadas a/T e o **fluxo físico** é recuperado por $\hat{x} \cdot T$ no final.
- **Early stopping**: o CLEAN com PSF aproximada acaba por super-ajustar — guardamos o modelo de **menor** $\|\delta v\|^2$ e paramos quando ele piora.
- Dirty beam e taper computados **por faceta** (mesma resolução do imageamento por faceta).
- As fontes ficam no **campo útil de cada faceta** (perto do centro da faceta) — que é exatamente a motivação do faceamento.

Resultado: recuperação estável (posição de pico exata, $\|\delta v\|^2$ caindo ~ 4 ordens de grandeza). Com faceamento as fontes ficam centradas nas facetas e a apodização é mais suave: $I=1$ recupera 84–106%,

`I=2x2` recupera 98–102%, `I=4x4` recupera ~100%.

8. Como foi implementado em C++

Arquivos

```
ddfacet_cpp/
├── include/ddfacet.h    ← estruturas de dados + declarações + pseudocódigo comentado
├── include/ms_io.h      ← interface do leitor de Measurement Set
├── src/fft.cpp          ← FFT/IFFT 2D via FFTW3, fftshift/fftshift
├── src/ddfacet.cpp      ← operadores: initialization, imaging_fft(_inv),
                        │   degrid, grid, compute_residual, deconvolution
├── src/ms_io.cpp        ← leitor de MS real (casacore)
├── src/main.cpp         ← geração/leitura do MS, dirty beam, loop principal, validação
└── tools/make_ms.py     ← gerador de MS real (python-casacore)
```

Estruturas de dados principais (`include/ddfacet.h`)

- `Array2D<T>` — grade 2D *row-major*; `operator()(i,j) = data[j*nx + i]` (`i` = coluna, `j` = linha). Aliases: `ImageF` (`float`, imagens) e `GridC` (`complex<float>`, grades UV).
- `Visibility` / `VisibilitySet` — as visibilidades (colunas do MS).
- `Facet` — região da imagem com grade UV e centro (`lo`, `mo`) próprios.
- `DDFacetConfig` — parâmetros: `K` (major cycles), `I = nx·ny` facetas, tamanho da imagem, `cell_size_rad`, e parâmetros do CLEAN (`clean_gain`, `n_minor_iterations`, `clean_threshold`).
- `DDFacetState` — estado global: `x` (modelo), `delta_y` (residual), `psf`, `facets`, `measurement_sets`.

FFT (`src/fft.cpp`)

Usa **FFTW3** (precisão dupla internamente). `fft_2d(grid, forward)` usa `fftw_plan_dft_2d(ny, nx, ...)` (linha = `n0`); a IFFT é normalizada por `1/(nx·ny)`. `fftshift_2d` / `ifftshift_2d` trocam os quadrantes para centrar o DC. *(Este arquivo não é modificado pela paralelização — FFTW permanece serial nesta fase.)*

Convenções importantes

- Pixel (`ix, iy`) da grade (após `fftshift`) ↔ coordenada UV `u = (ix - nx/2)/(nx·cell)`.
- Conversão inversa (UV → pixel): `ix_c = u·nx·cell + nx/2`.
- Kernel de gridding: `w_support = 5`, `σ = 1` pixel.

9. Paralelização (OpenMP)

Paralelização **de um nó** (memória compartilhada) com **OpenMP**, em **dois eixos**:

(a) Eixo das FACETAS (`I`) — grão grosso, o eixo principal. O laço de facetas no `main` usa `#pragma omp parallel for if(I>1)`: cada faceta é independente. Para evitar corrida no `degrid` (que somava em `predicted` compartilhado), **cada faceta escreve no seu próprio buffer** (`facet.pred_contrib`) e o `main` soma as contribuições depois. Com `I=1`, o `if(I>1)` desliga este nível e o paralelismo interno (b) é que atua.

(b) Eixo interno (visibilidades / pixels) — grão fino, atua quando `I=1` :

Operador	Estratégia	Justificativa
<code>degrid</code>	<code>#pragma omp parallel for</code>	cada visibilidade <code>k</code> escreve índice exclusivo → data-parallel
<code>grid</code>	grade local por thread + redução (crítica)	visibilidades vizinhas escrevem nas mesmas células → corrida. Mesmo padrão que vira <i>*all-reduce*</i> MPI no multi-nó
<code>imaging_fft</code> / <code>imaging_fft_inv</code>	<code>#pragma omp parallel for collapse(2)</code>	cada pixel escreve célula exclusiva
<code>deconvolution</code>	serial	CLEAN é sequencial: cada iteração depende da subtração anterior

Armadilha resolvida: o **planejador do FFTW não é thread-safe**. Ao paralelizar as facetas, várias threads chamam `fftw_plan_dft_2d` ao mesmo tempo → corrupção/crash. Solução: `fftw_make_planner_thread_safe()` uma vez no início (link `-lfftw3_threads`), sem tocar no `fft.cpp`.

Resultado — eixo interno, `I=1` (CPU i7-10750H, 6 físicos / 12 lógicos):

Threads	1	2	4	6	8	12
Tempo (ms)	572	321	215	170	144	219
Speedup	1.0×	1.78×	2.66×	3.36×	3.96×	2.61×

Resultado — eixo das FACETAS, `I=4x4` (16 facetas):

Threads	1	2	4	6
Tempo (ms)	5092	2760	1566	1187
Speedup	1.0×	1.84×	3.25×	4.29×

Pontos de discussão (para um orientador de paralelismo):

- **Correção preservada:** o resultado é **idêntico** em qualquer nº de threads.
- **Speedup sublinear (Lei de Amdahl):** a fração serial (FFT do FFTW, CLEAN, setup, seção crítica do `grid`) limita o ganho. No eixo interno (`I=1`) a fração serial dá teto $\sim 6.9\times$.
- **Regressão em 12 threads:** a CPU tem só **6 núcleos físicos**; acima disso o **hyperthreading** compete pelas mesmas unidades de execução e o overhead domina.
- **Faceamento custa mais trabalho total** (cada faceta processa todas as visibilidades) — daí `I=4x4` levar ~ 5 s vs ~ 0.5 s do `I=1` — **mas é justamente esse trabalho extra que é altamente paralelo** pelo eixo das facetas.

10. Como rodar e validar

No terminal **Ubuntu (WSL)**:


```
cd "/mnt/c/Users/thiag/OneDrive/Área de Trabalho/DDFacet cpp/ddfacet_cpp"

# Compilar (OpenMP + FFTW-threads + casacore)
g++ -std=c++17 -O2 -Wall -fopenmp -Iinclude -isystem /usr/include/casacore \
    src/ddfacet.cpp src/fft.cpp src/main.cpp src/ms_io.cpp \
    -lfftw3_threads -lfftw3 -lm -lcasa_ms -lcasa_tables -lcasa_casa -o build/ddfacet_demo

# Recuperação validada (I=1), lendo o MS real (gerado na 1ª vez)
OMP_NUM_THREADS=6 ./build/ddfacet_demo

# Faceamento I=2x2: recupera 98-102%
DDF_FACETS=2 OMP_NUM_THREADS=6 ./build/ddfacet_demo

# Escalabilidade do eixo INTERNO (I=1)
for t in 1 2 4 6 8; do echo "=== $t thread(s) ==="; \
    OMP_NUM_THREADS=$t ./build/ddfacet_demo 2>/dev/null | grep -E "Tempo do loop|RESULTADO"; done

# Escalabilidade do eixo das FACETAS (16 facetas)
for t in 1 2 4 6; do echo "=== $t thread(s) ==="; \
    DDF_FACETS=4 OMP_NUM_THREADS=$t ./build/ddfacet_demo 2>/dev/null | grep "Tempo do loop"; done
```

Alternativa com CMake: `mkdir -p build && cd build && cmake .. && make` → `./ddfacet_cpp`. `DDF_FACETS=N` define `N×N` facetas (padrão 1).

O que observar na saída:

- Testes de FFT 1D/2D: ✓ PASS (round-trip correto).
- `[read_ms]` ... : leitura do MS real (nº de visibilidades, frequência).
- Por major cycle: $\|\delta v\|^2$ caindo (convergência).
- Validação final: cada fonte recuperada (~100%), pico no lugar certo, `RESULTADO: ✓ Fontes recuperadas com sucesso`.
- `Tempo do loop principal` + nº de threads (para o speedup).

11. Próximos passos

Já feito nesta fase: **faceamento** `I>1` **ponta a ponta, paralelização por facetas (OpenMP) e leitura de MS real (casacore)**. A seguir:

1. **MPI multi-nó** sobre os Measurement Sets (`J`) — o `grid` já está no formato de redução por threads, que vira naturalmente um **all-reduce** entre nós.
2. **Threads do FFTW** (`fftw_plan_with_nthreads`) — ataca a maior fatia serial restante.
3. **W-term** (baselines não-coplanares) e **MSMF** (deconvolução multi-frequência).
4. **Overlap entre facetas** + reprojeção (facetadas com sobreposição para evitar costuras).

Referência: Monnier et al., "Análise e implementação de um pipeline de imageamento para rádio-interferometria", SiPS IEEE 2022. Implementação C++ educacional – fase sequencial + OpenMP.